

Министерство Образования Российской Федерации
УрФУ
Образовательных информационных технологий

Лабораторная работа №7
Тема: «Основы работы с Redis»

Выполнил:
Нарсеев А.В.

Екатеринбург, 2025

Лабораторная работа №7: Основы работы с Redis

Цель работы: овладеть базовыми навыками установки, подключения и взаимодействия с Redis в Python. Изучить основные структуры данных Redis и их применение на практике.

Добавление контейнера Redis

Добавим контейнер для Redis:

```
▷ Run Service
redis:
  image: redis:latest
  container_name: redis
  ports:
    - "6379:6379"
  volumes:
    - redis-data:/data
  networks:
    - lr7_network

volumes:
  postgres_data:
  rabbitmq_data:
  redis-data:
```

Подключение к Redis

Попробуем подключиться к контейнеру

```
import redis

# Подключение к локальному Redis
r = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True)

# Проверка подключения
try:
    r.ping()
    print("Успешное подключение к Redis")
except redis.ConnectionError:
    print("Ошибка подключения к Redis")
```

Можем проверить как работает redis с разными типами данных

Работа со строками

```

# Работа со строками
print("\n=== Работа со строками ===")
r.set("user:name", "Иван")
name = r.get("user:name")
print(f"Имя пользователя: {name}")

# Установка с TTL
r.setex("session:123", 3600, "active") # 1 час
print(f"Сессия: {r.get('session:123')}")

```

Работа с числами

```

# Работа с числами
r.set("counter", 0)
r.incr("counter") # Увеличить на 1
r.incrby("counter", 5) # Увеличить на 5
r.decr("counter") # Уменьшить на 1
print(f"Счетчик: {r.get('counter')}")

```

Работа со списками

```

# Работа со списками
print("\n=== Работа со списками ===")
r.lpush("tasks", "task1", "task2") # В начало
r.rpush("tasks", "task3", "task4") # В конец
tasks = r.lrange("tasks", 0, -1) # Все элементы
print(f"Все задачи: {tasks}")
first_task = r.lpop("tasks") # Удалить и вернуть первый
last_task = r.rpop("tasks") # Удалить и вернуть последний
print(f"Первая задача: {first_task}, Последняя задача: {last_task}")
length = r.llen("tasks")
print(f"Длина списка: {length}")

```

Работа с множествами

```

# Работа с множествами
print("\n=== Работа с множествами ===")
r.sadd("tags", "python", "redis", "database")
r.sadd("languages", "python", "java", "javascript")
is_member = r.sismember("tags", "python") # True
print(f"Python в тегах: {is_member}")
all_tags = r.smembers("tags")
print(f"Все теги: {all_tags}")
intersection = r.sinter("tags", "languages") # Пересечение
union = r.sunion("tags", "languages") # Объединение
difference = r.sdiff("tags", "languages") # Разность
print(f"Пересечение: {intersection}")
print(f"Объединение: {union}")
print(f"Разность: {difference}")

```

Работа с хэшами

```
# Работа с хэшами
print("\n=== Работа с хэшами ===")
r.hset("user:1000", mapping={
    "name": "Иван",
    "age": "30",
    "city": "Москва"
})
name = r.hget("user:1000", "name")
all_data = r.hgetall("user:1000")
print(f"Имя: {name}")
print(f"Все данные: {all_data}")
exists = r.hexists("user:1000", "email")
print(f"Email существует: {exists}")
keys = r.hkeys("user:1000")
values = r.hvals("user:1000")
print(f"Ключи: {keys}")
print(f"Значения: {values}")
```

Работа с упорядоченными множествами

```
# Работа с упорядоченными множествами
print("\n=== Работа с упорядоченными множествами ===")
r.zadd("leaderboard", {
    "player1": 100,
    "player2": 200,
    "player3": 150
})
top_players = r.zrange("leaderboard", 0, 2, withscores=True)
print(f"Топ игроки: {top_players}")
players_by_score = r.zrangebyscore("leaderboard", 100, 200)
print(f"Игроки по очкам (100-200): {players_by_score}")
rank = r.zrank("leaderboard", "player1")
print(f"Ранг player1: {rank}")

print("\nВсе тесты выполнены успешно!")
```

Кэширование запросов и очищение запросов

После ознакомления с библиотекой `redis` мы можем добавить в приложение кэширование. Для получения данных о пользователе необходимо закэшировать данные о нём и хранить в течение часа, при обновлении данных пользователя из кэша данные удаляем.

Получим пользователя из БД и закэшируем его:

```
(venv) narseev@Mac LR7 % curl "http://localhost:8000/users/9bf54a6a-3693-4d18-a34a-dd7b0f5890a6" | jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left  Speed
100    230    100    230      0      0    39525      0 --:--:-- --:--:-- --:--:-- 46000
{
  "id": "9bf54a6a-3693-4d18-a34a-dd7b0f5890a6",
  "username": "alice",
  "email": "alice@example.com",
  "description": "Активный покупатель",
  "created_at": "2025-12-08T18:17:16.610061",
  "updated_at": "2025-12-08T18:17:16.610063"
}
```

Проверим, что пользователь закэширован и посмотрим TTL:

```
(venv) narseev@Mac LR7 % docker exec -it redis redis-cli GET "user:9bf54a6a-3693-4d18-a34a-dd7b0f5890a6"
"{\"id\": \"9bf54a6a-3693-4d18-a34a-dd7b0f5890a6\", \"username\": \"alice\", \"email\": \"alice@example.com\", \"description\": \"Активный покупатель\", \"created_at\": \"2025-12-08 18:17:16.610061\", \"updated_at\": \"2025-12-08 18:17:16.610063\"}"
(venv) narseev@Mac LR7 % docker exec -it redis redis-cli TTL "user:9bf54a6a-3693-4d18-a34a-dd7b0f5890a6"
(integer) 3547
```

При работе с продукцией, также производим кэширование данных с ограничением в 10 минут, при обновлении данных необходимо обновить данные в кэше.

Аналогично с продукцией

```
(venv) narseev@Mac LR7 % curl "http://localhost:8000/products" | jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left  Speed
100    998    100    998      0      0   106k      0 --:--:-- --:--:-- --:--:-- 108k
{
  "total": 6,
  "page": 1,
  "count": 10,
  "products": [
    {
      "id": "425ec6ef-12a1-4735-ac5b-0a5d97e207ca",
      "name": "Ноутбук Aurora 15",
      "price": 79900.0,
      "stock_quantity": 7,
      "created_at": "2025-12-08T18:17:16.613008"
    },
    {
      "id": "625206be-20a6-4823-87f2-b95ae30d901e",
      "name": "Смартфон Vega X",
      "price": 45990.0,
      "stock_quantity": 14,
      "created_at": "2025-12-08T18:17:16.613013"
    },
    {
      "id": "c570c86a-a971-4e11-b4d0-b0101a20834f",
      "name": "Наушники Pulse",
      "price": 8990.0,
      "stock_quantity": 22,
      "created_at": "2025-12-08T18:17:16.613016"
    },
    {
      "id": "160fec718-6e55-48e7-b021-7ded5ede7b9e",
      "name": "Наушники Pulse",
      "price": 8990.0,
      "stock_quantity": 22,
      "created_at": "2025-12-08T18:17:16.613016"
    },
    {
      "id": "160fec718-6e55-48e7-b021-7ded5ede7b9e",
      "name": "Наушники Pulse",
      "price": 8990.0,
      "stock_quantity": 22,
      "created_at": "2025-12-08T18:17:16.613016"
    },
    {
      "id": "160fec718-6e55-48e7-b021-7ded5ede7b9e",
      "name": "Наушники Pulse",
      "price": 8990.0,
      "stock_quantity": 22,
      "created_at": "2025-12-08T18:17:16.613016"
    }
  ]
}
```

Видно, что она закэширована (10 минут):

```

• (venv) narseev@Mac LR7 % docker exec -it redis redis-cli KEYS "products:list:*"
1) "products:list:page:1:count:10"
• (venv) narseev@Mac LR7 % docker exec -it redis redis-cli TTL "products:list:page:1:count:10"
(integer) 552

```

Далее получим один продукт по id посмотрим закеширован ли он и его TTL

```

• (venv) narseev@Mac LR7 % curl "http://localhost:8000/products/86b6b025-991a-4d79-83fe-98313d0ea288"
{"id":"86b6b025-991a-4d79-83fe-98313d0ea288","name":"Тестовый продукт","price":1000.0,"stock_quantity":10,"created_at":"2025-12-08T18:48:41.553176"}
• (venv) narseev@Mac LR7 % docker exec -it redis redis-cli GET "product:86b6b025-991a-4d79-83fe-98313d0ea288"
"{\"id\": \"86b6b025-991a-4d79-83fe-98313d0ea288\", \"name\": \"\u0422\u0435\u0441\u0442\u043e\u0432\u044b\u0439 \u043f\u0440\u043e\u0434\u0443\u043a\u0442\", \"price\": 1000.0, \"stock_quantity\": 10, \"created_at\": \"2025-12-08 18:48:41.553176\"}"
• (venv) narseev@Mac LR7 % docker exec -it redis redis-cli TTL "product:86b6b025-991a-4d79-83fe-98313d0ea288"
(integer) 562

```

TTL обновился

Вопросы

- В чем заключается основное преимущество хранения данных в оперативной памяти (in-memory) по сравнению с дисковыми БД?
Скорость доступа: операции в памяти на порядки быстрее, чем чтение/запись на диск
- Для чего нужен параметр `decode_responses=True` при создании клиента Redis?
Автоматическое декодирование: redis по умолчанию возвращает байты, а `decode_responses=True` автоматически декодирует их в строки
- Что такое TTL (Time To Live) ключа и как он используется в Redis?
TTL задает срок хранения ключа в секундах. По истечении ключ автоматически удаляется. Используется для кэширования с ограниченным временем жизни
- Объясните разницу между командами `lpush()` и `rpush()` для списков.
 - `lpush()` — добавляет элементы в начало списка (left push)
 - `rpush()` — добавляет элементы в конец списка (right push)
- Как обеспечить атомарность операций в Redis?
Транзакции (MULTI/EXEC), команды Lua-скриптов или специальные атомарные команды (например, INCR, SETNX, HINCRBY)
- Как в Redis реализована репликация и кластеризация?
Репликация: мастер-реплика (master-replica). Мастер принимает запись, реплики копируют данные для чтения и отказоустойчивости. Кластеризация: шардирование данных по узлам (hash slots). Автоматическое распределение нагрузки и обеспечение высокой доступности через репликацию внутри кластера.